CYBERSHIELD

Cybersecurity Awareness & Threat Detection System

Academic Micro-Project Report

A responsive educational web application for URL awareness, password hygiene, and practical cybersecurity learning.

Technology: Python 3 - Flask - SQLite - HTML - CSS - JavaScript

Report edition: 1.0 | Date: 01 November 2026

Project at a glance

CyberShield combines an interactive dashboard, a local rule-based URL syntax analyzer, a browser-only password assessment, and concise awareness content. The goal is to help learners recognize common warning signs while clearly communicating the limits of lightweight educational checks.

Abstract

The project demonstrates a small full-stack application built with Flask and SQLite. URL input is parsed as text and classified using transparent rules for transport scheme, IP literals, suspicious wording, hostname shape, and ports. No submitted URL is fetched or resolved. Password composition is inspected in the browser, and the password string is never sent to the server. Anonymous activity records retain event category, result category, and timestamp only. The project is designed for classroom demonstration, not operational defense.

1. Introduction

People routinely encounter deceptive links, credential theft attempts, and unsafe password habits. Cybersecurity awareness improves when learners can interact with simple examples and receive immediate, understandable feedback. CyberShield presents that learning in a responsive dashboard.

1.1 Problem statement

Beginners often struggle to distinguish suspicious URL patterns and to evaluate password composition. A small, safe demonstration can explain observable signals without opening a potentially harmful destination.

1.2 Objectives

- Provide an at-a-glance dashboard of usage totals.
- Explain why URL syntax may warrant caution.
- Assess sample password composition locally in the browser.
- Present practical advice on phishing and safer browsing.
- Store only minimal, non-secret activity aggregates in SQLite.

1.3 Scope and limitations

This is a teaching aid. It has no threat-intelligence feed, malware detection, reputation lookup, DNS inspection, or site-content inspection. URL rules can produce false positives and false negatives. -Safe- means only that the rules did not find a listed pattern. HTTPS encrypts a connection but does not prove that a destination is legitimate. Password score is a composition checklist, not a complete estimate of resistance to guessing or reuse.

2. Requirements

Functional: dashboard totals and recent events; URL input validation and a three-level classification with explanations; password checklist and tips; awareness cards; JSON endpoints; persistent local activity history.
Non-functional: responsive layout, beginner-friendly setup, minimal dependencies, no URL visitation, no password storage/transmission, and clear limitations.

3. System design

3.1 Architecture

The browser renders a Flask-served HTML page and static CSS/JavaScript. URL analysis is sent to Flask for local parsing and rule evaluation. Aggregate event metadata is written to SQLite. Password scoring stays in JavaScript; a separate empty POST increments an assessment count. The endpoint ignores request content.

```
Browser  -  Flask URL parser  -  SQLite event metadata
Browser JavaScript  -  empty count request  -  SQLite
```

3.2 URL analysis method

The analyzer accepts HTTP or HTTPS URLs, adding HTTPS when no scheme is given. It validates host presence, rejects whitespace and embedded credentials, and does not perform network I/O. Rule points are assigned for plain HTTP, direct IP address, uncommon suffix, long host, many hyphens, suspicious vocabulary, nonstandard port, and deeply nested subdomains. Score 0-1 is Safe, 2-5 is Suspicious, and 6+ is High Risk. The displayed score is capped at 10.

3.3 Password assessment and privacy

Five visible checks cover length of at least 12, uppercase, lowercase, digit, and special character; extra length can improve the score. Suggestions identify missing criteria. No password is placed in a request. The activity endpoint intentionally ignores its request body and writes only a generic count row.

3.4 Data model

`activity(id, kind, classification, created_at)`

The schema constraint allows only url_scan or password_check event kinds. URL text and password content are absent from the table by design.

4. Implementation

The application entry point is app.py. Flask serves the dashboard, health check, dashboard data, URL analysis, and password assessment count. Database setup is automatic. CYBERSHIELD_DB can select a custom SQLite path.
The interface is composed of templates/index.html, static/style.css, and static/app.js. Semantic labels, focus styling, responsive breakpoints, and an aria-live result area support usability across screen sizes.

4.1 API summary

```
GET  /                      Dashboard page
GET  /health                Liveness response
GET  /api/dashboard         Counts and recent activity
POST /api/analyze-url       Analyze JSON/form field named url
POST /api/password-assessment  Increment count; body ignored
```

4.2 Input handling

The server enforces a 2,048-character URL maximum, rejects whitespace and control characters, schemes other than HTTP(S), incomplete hostnames, malformed ports, and URLs containing credentials. JSON errors use HTTP 400. Flask also has a request-body size limit. Results are inserted into the page using DOM text nodes, rather than interpreted HTML.

5. Testing and verification plan

The unittest suite covers the landing page, health endpoint, safe and suspicious classification, validation errors, aggregate dashboard counts, and confirming that URL/password input strings are absent from persisted event records.
Run with: python -m unittest discover -s tests -v
Run locally: python -m venv .venv; activate it; pip install -r requirements.txt; then python app.py and visit http://127.0.0.1:5000.

Environment note: this report was authored in an environment without a Python runtime, so application tests and live endpoint checks could not be executed here.

6. Deployment and operation

For local demonstration, use a supported Python 3 version, install Flask from requirements.txt, and launch app.py. The server defaults to loopback-friendly Flask development settings. The generated SQLite file lives in instance/ and is excluded from version control. Tests create isolated temporary databases.

6.1 Production considerations

Before any public deployment, use a production WSGI server, HTTPS, rate limits, security headers, operational monitoring, backup/retention decisions, and a reviewed privacy notice. The educational URL score must not be presented as a professional security verdict.

7. Conclusion

CyberShield joins foundational awareness content with hands-on URL and password exercises. The implementation emphasizes transparent rule explanations, local analysis, minimal history, and explicit privacy boundaries. It is a useful starting point for a college micro-project and can be extended with additional lessons or accessibility refinements without changing its safe-by-default educational scope.

Appendix A. Suggested demonstration

1. Open the dashboard and note the empty or accumulated counts.
2. Analyze https://example.com and review the basic rule result.
3. Analyze http://192.168.1.3/urgent-login to see multiple warning signals.
4. Enter a sample password and inspect the checklist; use a fake sample only.
5. Confirm that only count metadata is retained in the activity database.

Appendix B. Repository contents

```
app.py, templates/index.html, static/app.js, static/style.css
tests/test_app.py, requirements.txt, README.md, LICENSE, .gitignore
report/CyberShield_Project_Report.pdf
```